

## บทที่ 5

### ฐานข้อมูลเชิงวัตถุ

แนวความคิดเขียนโปรแกรมแบบออบเจกต์เป็นรูปแบบใหม่ในการจัดการกับการกำหนดสิ่งต่างๆ ให้ใกล้เคียงกับสิ่งต่างๆ ในโลกความเป็นจริงที่เรียกว่า ออบเจกต์ แนวความคิดเชิงวัตถุเริ่มมีใช้ในภาษาโปรแกรมมาก่อน เช่น ภาษาจาวา(Java) และภาษาซีพลัสพลัส (C++) สำหรับแบบจำลองที่นำเสนอโครงสร้างข้อมูลในแนวทางออบเจกต์เรียกว่า แบบจำลองข้อมูลแบบออบเจกต์ ฐานข้อมูลที่ใช้แบบจำลองข้อมูลซึ่งแสดงข้อมูลและความสัมพันธ์ระหว่างข้อมูลในแนวทางของออบเจกต์ เรียกว่า ฐานข้อมูลเชิงวัตถุ(Object-Oriented Databases: OODB)

ในบทนี้ได้เปรียบเทียบถึงความแตกต่างระหว่างเทคโนโลยีแบบออบเจกต์กับเทคโนโลยีแบบรีเลชัน แนวความคิดของฐานข้อมูลแบบเชิงสัมพันธ์ และแนวความคิดของฐานข้อมูลเชิงวัตถุ อยู่ในส่วนของ 1. และ 2. ตามลำดับ

#### 5.1. แนวความคิดของฐานข้อมูลแบบเชิงสัมพันธ์ (RDB)

ฐานข้อมูลแบบเชิงสัมพันธ์จะนำเสนอเป็นรูปแบบของตาราง โดยประกอบด้วยแถวและคอลัมน์ ซึ่งเป็นมุมมองแบบลอจิกคัลของฐานข้อมูลเพื่อแสดงให้ผู้ใช้งานและโปรแกรมเมอร์ดู ส่วนข้อมูลที่เก็บอยู่ในระบบคอมพิวเตอร์นั้นจะเรียกว่ามุมมองแบบฟิสิกัล ฟิวด์ในเรคอร์ดจะเรียกว่าแอททริบิวต์ แต่ละคอลัมน์จะมีชนิดของข้อมูล เช่น ข้อมูลชนิดตัวเลข ข้อมูลชนิดอักขระ ข้อมูลชนิดวันที่ เป็นต้น ซึ่งก่อนที่จะมีการเก็บข้อมูลลงฐานข้อมูลจะต้องมีการตรวจสอบความถูกต้องข้อมูลก่อนเรียกว่า เงื่อนไข มีทั้งเงื่อนไขแบบโดเมน, เงื่อนไขแบบคีย์, เงื่อนไขแบบเอนทิตีอินทิกริตี (Entity integrity) และเงื่อนไขแบบเรฟเฟอเรนเชียลอินทิกริตี (Referential integrity) ซึ่งทำให้แน่ใจได้ว่าไม่มีข้อมูลที่ผิดปกติอยู่ในฐานข้อมูล

ฐานข้อมูลแบบเชิงสัมพันธ์ใช้ภาษาเอสคิวแอล (Structure Query Language : SQL) เป็นภาษาคำสั่งนิยาม (Data Definition : DDL) หรือภาษาคำดำเนินการ (Data Manipulation : DML) ซึ่งเอสคิวแอลนั้นจะรวมคำสั่งที่ใช้ในการกำหนดชนิดของข้อมูล, คำสั่งที่ใช้ในการเปลี่ยนแปลงข้อมูล, คำสั่งคิวรี และคำสั่งที่ใช้กำหนดเงื่อนไข การคิวรีนั้นจะแบ่งตามลักษณะของการคิวรี ซึ่งเป็นมาตรฐานเดียวกันของภาษาเอสคิวแอลทำให้ผู้ใช้งานง่ายต่อการย้ายดาต้าเบส แอปพลิเคชันข้ามระบบฐานข้อมูลของแต่ละบริษัทได้ ตัวอย่างเช่น ผู้ใช้สามารถเข้าถึงข้อมูลที่เก็บไว้แต่ละฐานข้อมูลได้โดยไม่ต้องเปลี่ยนแปลงคำสั่งของเอสคิวแอล ข้อดีอีกอย่างคือการเข้าถึงข้อมูลได้เร็วแม้ว่าฐานข้อมูลจะมีขนาดใหญ่ ข้อเสียหลักของฐานข้อมูลแบบเชิงสัมพันธ์ คือ ไม่สามารถเก็บข้อมูลประเภทรูปภาพได้ ไม่สามารถใช้ชนิดข้อมูลแบบพิเศษได้ เช่น จำนวนเชิงซ้อน, อาร์เรย์ และแอปพลิเคชันที่มีการเชื่อมโยงของข้อมูลที่ซับซ้อน (Elmasri และ Navathe 2000)

| time_key<br>(CHAR(10)) | fiscal_year<br>(NUMBER(4)) | fiscal_quarter<br>(CHAR(10)) | Fiscal_month<br>(CHAR(10)) |
|------------------------|----------------------------|------------------------------|----------------------------|
| 1999Q1M01              | 1999                       | Quarter1                     | January                    |
| 1999Q1M02              | 1999                       | Quarter1                     | February                   |
| 1999Q1M03              | 1999                       | Quarter1                     | March                      |
| 2000Q1M10              | 2000                       | Quarter4                     | October                    |
| 2000Q1M11              | 2000                       | Quarter4                     | November                   |
| 2000Q1M12              | 2000                       | Quarter4                     | November                   |
| 2001Q1M01              | 2001                       | Quarter1                     | January                    |
| 2001Q1M02              | 2001                       | Quarter1                     | February                   |
| 2001Q1M03              | 2001                       | Quarter1                     | March                      |
| 2001Q1M04              | 2001                       | Quarter?                     | ...April...                |
| 2002Q1M01              | 2002                       | Quarter1                     | January                    |
| 2002Q1M02              | 2002                       | Quarter1                     | February                   |
| 2002Q1M03              | 2002                       | Quarter1                     | March                      |
| 2002Q1M04              | 2002                       | Quarter2                     | April                      |
| 2002Q1M05              | 2002                       | Quarter2                     | May                        |
| 2002Q1M06              | 2002                       | Quarter2                     | June                       |

ตารางที่ 5.1 แสดงตัวอย่าง ฐานข้อมูลแบบเชิงสัมพันธ์

```

select * from films where studio = "MGM"
      into MGM_films ;

select  title, year from parts
      where year < 1940 & ~(role = "Armand")
      into  early_films

select  name, films.title, films.year, role
from    films, parts
where   films.title = parts.title &
        films.year = parts.year &
        studio = "MGM"
into    MGM_stars

```

ตัวอย่าง แสดงคำสั่งคิวรีด้วยภาษาเอสคิวแอล

## 5.2. แนวความคิดของฐานข้อมูลแบบเชิงวัตถุ (OODB)

มีลักษณะการเก็บข้อมูลและการเรียกใช้ข้อมูลที่มีรูปแบบแตกต่างจากระบบฐานข้อมูลแบบเชิงสัมพันธ์ คือจะเป็นการมองข้อมูลที่ถูกจัดเก็บในรูปแบบของวัตถุ(Object) ซึ่งวัตถุจะประกอบด้วยแอตทริบิวต์ และเมธอด เราสามารถอธิบายลักษณะโดยรวมที่สำคัญๆ ของระบบข้อมูลเชิงวัตถุได้ ดังนี้

### 5.2.1. Object Identity

จะสนับสนุนไเดนทิตี(Identity) ของ Object Instance โดยไม่สนใจว่าตัวแปร(Attribute) ของแต่ละออปเจกต์ จะมีค่าเท่าไร และแต่ละออปเจกต์ต้องมีค่า Object Identity เพื่อไว้ใช้อ้างอิงไปยังออปเจกต์ตัวอื่นในแอปพลิเคชัน สำหรับในระบบฐานข้อมูลที่เป็นแบบเน้นการแสดงค่า(Value-Oriented) จะใช้ค่า หนึ่งในที่อยู่ในฐานข้อมูลเป็นค่าหลัก(Key) ในการแทนถึงวัตถุใดวัตถุหนึ่ง ปัญหาที่เกิดขึ้นจากการทำวิธีนี้คือ เมื่อค่าหลักที่ใช้

ในการแทนออบเจกต์มีการเปลี่ยนแปลง จะทำให้เกิดปัญหาการอ้างถึงวัตถุที่ผิดไปจากตัวเดิมได้ และต้องแก้ปัญหาดังกล่าวด้วยการแก้ไขค่าหลักที่ใช้ทำการอ้างอิงนี้ทุกตัวเมื่อมีการเปลี่ยนแปลงเกิดขึ้น

ในกรณีของระบบฐานข้อมูลเชิงวัตถุ จะแก้ปัญหานี้ด้วยการแทนค่าที่ใช้ในการอ้างอิงถึงตัวออบเจกต์ได้นั้นเป็นค่าต่างหากค่าหนึ่ง ซึ่งเราเรียกค่านั้นว่า Object Identifier โดยค่านี้นี้จะแตกต่างจากค่าที่เก็บอยู่ในฐานข้อมูลค่าอื่นๆ คือ จะมีค่าที่ไม่ซ้ำกัน ซึ่งค่า Object Identifier ระบบฐานข้อมูลจะใช้ในการอ้างอิงถึง ออบเจกต์เป็นแบบถาวร คือ ค่าที่ใช้ในการอ้างอิงถึงออบเจกต์เหล่านั้นจะถูกสร้าง และทำลายโดยตัวระบบจัดการฐานข้อมูลเอง และเราไม่สามารถทำการเปลี่ยนแปลงค่าต่างๆ เหล่านั้นได้

### 5.2.2. Complex Objects

Complex objects คือออบเจกต์ที่ถูกสร้างมาจากออบเจกต์พื้นฐานหลายๆ ตัว ด้วยการประยุกต์วิธีการสร้างให้กับออบเจกต์เหล่านั้น สำหรับออบเจกต์แต่ละตัวจะมีค่าที่เกี่ยวข้องด้วยอยู่ค่าหนึ่ง ซึ่งอาจเป็นค่าที่มีลักษณะซับซ้อนก็ได้ ซึ่งค่านี้อาจเป็นค่าของ Object Identifier ก็ได้ เป็นผลให้สามารถมีออบเจกต์สองตัวใดๆ ที่อ้างถึงออบเจกต์อีกตัวหนึ่งซึ่งเป็นตัวเดียวกันได้ โดยใช้ Object Identifier ที่มีค่าเหมือนกัน และเป็นผลให้เมื่อมีการเปลี่ยนแปลงข้อมูลในตัวออบเจกต์นั้นๆ ก็จะไม่ส่งผลกระทบไปยังออบเจกต์ตัวอื่นๆ ที่มีการอ้างอิง ซึ่งค่าที่เก็บอยู่ในออบเจกต์นี้สามารถเรียกได้อีกอย่างว่าเป็นสถานะ(State) ของออบเจกต์ตัวนั้นได้ด้วย Complex Datatype คือข้อมูลที่ซับซ้อน ซึ่งประกอบด้วยการนำเอา Simple Datatype(ประกอบไปด้วย integers, characters, byte string of any length, booleans และ floats) มารวมกันเป็นโครงสร้างของ datatype ตัวใหม่

### 5.2.3. Method

เมธอด คือ วิธีปฏิบัติการกับข้อมูล ซึ่งแต่ละคลาสจะมีเมธอดของตนเอง โดยการทำงานจะส่งข่าวสาร (Message) ไปยังออบเจกต์ที่ต้องการเพื่อเรียกใช้เมธอดที่อยู่ภายในออบเจกต์นั้น ออบเจกต์ที่เก็บอยู่ในระบบฐานข้อมูลเชิงวัตถุจะมีวิธีการใช้งานโดยผ่านทางเมธอดนี้ โดยเมธอดจะประกอบด้วยชื่อ รูปแบบในการใช้งาน (Signature) และส่วนที่เป็นเนื้อหาของเมธอด

### 5.2.4. Class

Class คือ กลุ่มของออบเจกต์ที่มีคุณสมบัติพื้นฐานเหมือนกัน การติดต่อกับคลาสกระทำผ่านเมธอดภายในคลาส ข้อมูลที่เก็บอยู่ภายในคลาสเรียกว่า เนื้อข้อมูล (instances) สำหรับออบเจกต์ใดๆ ที่เป็นของคลาสใดคลาสหนึ่งที่อยู่ฐานข้อมูลจะเรียกคลาสนั้นว่าเป็นคลาสหลัก (Primary Class, Immediate Class, Proper Class) ของออบเจกต์นั้นๆ ซึ่งจะเป็นคลาสที่ออบเจกต์ตัวนั้นมีความสัมพันธ์ในรูปแบบที่เป็น instance-of ของคลาสนั้น

### 5.2.5 Encapsulation

แนวความคิดของ Encapsulation มาจาก 1) ความจำเป็นในการแยกส่วนรายละเอียด และส่วนเรียกใช้งานของตัวออบเจกต์ 2) ความจำเป็นสำหรับการทำเป็นหน่วยย่อย Encapsulation คือการรวมกันของโครงสร้าง

ตัวแปรและเมธอด เกิดเป็นอ็อปเจกต์ใหม่ที่มีความสามารถในการซ่อนข้อมูลจากระบบภายนอกได้ ทำให้ข้อมูลมีความปลอดภัยขึ้น ซึ่งวิธีการป้องกันนั้นจะอยู่ที่ชนิดของการประกาศข้อมูลภายในคลาส จะมีอยู่ 3 ระดับ

1. **Private** เป็นการประกาศข้อมูลแบบที่ใช้ภายในตัวเอง จะมีผลทำให้ไม่มีคำสั่งใดจากภายนอกสามารถเข้าถึงข้อมูลภายในคลาสนั้นได้เลย นอกจากเมธอดภายในคลาสนั้นเอง การเรียกใช้ก็เรียกผ่านเมธอดของคลาส
2. **Public** เป็นการประกาศข้อมูลแบบสาธารณะ คือมีผลทำให้คลาสอื่นๆ หรือเมธอดอื่นๆ สามารถเข้าถึงข้อมูลภายในคลาสนี้ได้อย่างอิสระ
3. **Protected** เป็นการประกาศข้อมูลแบบสงวนไว้ให้คลาสลูก จะมีผลทำให้มีเฉพาะเมธอดภายในของคลาสนั้นและคลาสที่สืบทอด(Inheritance) เท่านั้นที่สามารถเข้าถึงข้อมูลภายในคลาสได้

#### 5.2.6. Inheritance

ในระบบฐานข้อมูลเชิงวัตถุ แนวคิดของการสืบทอดคุณสมบัติทำให้อ็อปเจกต์มีตัวแปร และเมธอดที่เจาะจง(Specialized) ลงไปจากคลาสที่เป็นคลาสต้นแบบ (Superclass) ได้ โดยเราจะเรียกคลาสที่ทำการสืบทอดคุณสมบัติมาจากคลาสต้นแบบว่าคลาสลูก (Subclass) ประโยชน์ของวิธีการสืบทอดคุณสมบัติคือ เราสามารถเพิ่มศักยภาพทางด้านการนำกลับมาใช้ใหม่ (Reusability) และการขยายเพิ่มเติมคุณสมบัติ (Extensibility) ได้

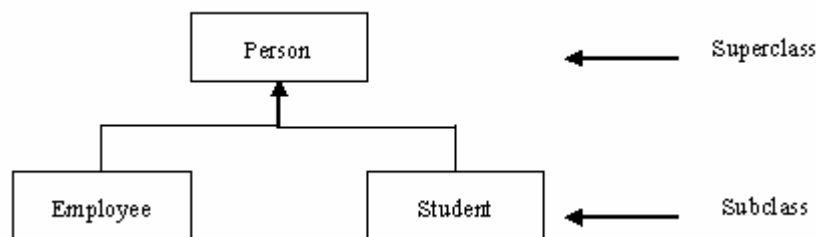
#### 5.2.7. Polymorphism

วิธีการ Polymorphism คือรูปแบบหนึ่งของการสืบทอดคุณสมบัติ โดยแบ่งเป็น 2 ลักษณะคือ 1) Overloading หมายถึงการที่มีเมธอดสองตัวใดๆ ในคลาสเดียวกันที่มีชื่อเหมือนกัน แต่มีรูปแบบการใช้งานต่างกัน 2) Overriding หมายถึงวิธีการที่อ้างถึงตัวแปร หรือเมธอดที่มีชื่อเหมือนกัน แต่มีการเปลี่ยนแปลงส่วนเนื้อหาของเมธอดที่มีการสืบทอดค่าคุณสมบัตินั้นมาจากคลาสต้นแบบ

#### 5.2.8. Relationship

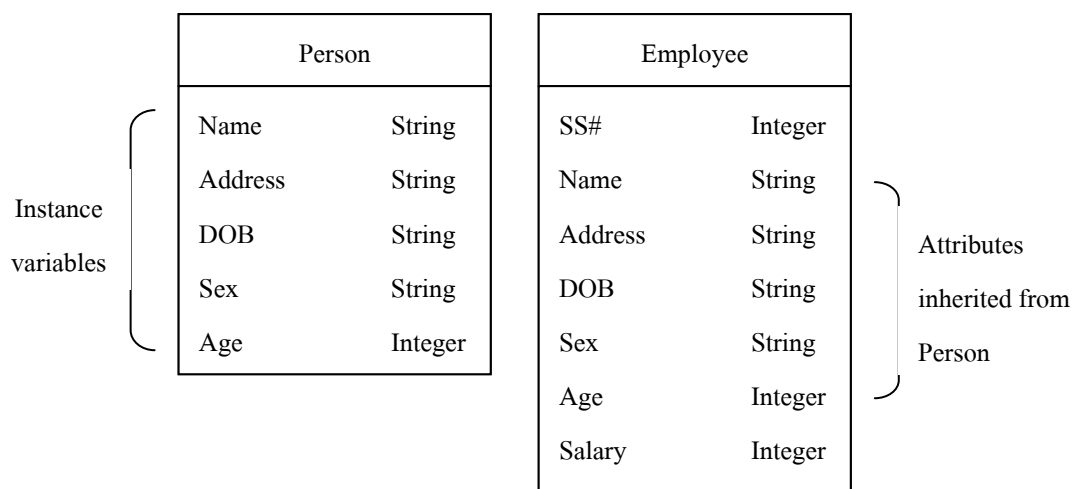
ความสัมพันธ์ในระบบฐานข้อมูลเชิงวัตถุมี 2 แบบหลักๆ คือ 1) Class-subclass relationship 2) Interobject relationship ซึ่งมี 3 รูปแบบคือ one-to-one, one-to-many และ many-to-many

- 1) **Class-subclass relationship** เป็นความสัมพันธ์ระหว่างคลาสที่อยู่ในรูปแบบลำดับชั้น ตัวอย่างรูปที่ 5-1 เป็นการแสดงความสัมพันธ์ของ Class-subclass โดยมีคลาส Person เป็นคลาสที่อยู่ระดับเหนือกว่า เรียกว่า Superclass และ Employee กับ Student เป็นคลาสที่อยู่ระดับต่ำกว่า เรียกว่า Subclass จากความสัมพันธ์แสดงให้เห็นว่า คลาส Employee กับ Student ต่างก็เป็น Subclass ของ Superclass Person



รูปที่ 5.1 ความสัมพันธ์ระหว่างคลาสในรูปแบบลำดับชั้น

โดยลักษณะความสัมพันธ์ระหว่างคลาสในรูปแบบ Class-subclass relationship นี้สนับสนุนหลักการของการถ่ายทอดคุณสมบัติ (inheritance) โดยคลาสที่อยู่ระดับสูงกว่าจะถ่ายทอดคุณสมบัติทุกอย่างมายังคลาสที่อยู่ระดับต่ำกว่า และ Subclass สามารถมีคุณสมบัติเพิ่มเติมนอกเหนือจากคุณสมบัติที่ได้รับการถ่ายทอดมาจาก Superclass ตัวอย่างรูปที่ 5-1 แสดงว่าคลาส Employee ได้รับการถ่ายทอดคุณสมบัติมาจากคลาส Person โดยที่ตัวแปร Name, Address, DOB, Sex และ Age ถ่ายทอดมาจาก Superclass Person ส่วน Subclass Employee นั้นมีตัวแปรเพิ่มขึ้นมาอีก 2 ตัวแปร คือ SS# และ Salary



รูปที่ 5.2 Subclass Employee ถ่ายทอดคุณสมบัติมาจาก Superclass Person

2) **Interobject relationship** เป็นความสัมพันธ์ระหว่างออบเจกต์ โดยมี 3 รูปแบบ

- Many-to-many relationship เป็นความสัมพันธ์จากคลาส C ไปคลาส D โดยความสัมพันธ์เป็นเซตของ D ที่มีความเกี่ยวข้องกับแต่ละออบเจกต์ของ C ในทางกลับกัน inverse relationship ก็เป็นเซตของ C ที่มีความเกี่ยวข้องกับแต่ละออบเจกต์ของ D

- One-to-many relationship เป็นความสัมพันธ์จากคลาส C ไปคลาส D โดยความสัมพันธ์สำหรับแต่ละ C จะได้ค่าที่ไม่ซ้ำกันของ D ในทางกลับกัน inverse relationship จะมีเซตของ C ที่มีความเกี่ยวข้องกับแต่ละออปเจกของ D

- One-to-one relationship เป็นความสัมพันธ์จากคลาส C ไปคลาส D โดยความสัมพันธ์สำหรับแต่ละ C จะได้ค่าที่ไม่ซ้ำกันของ D ในทางกลับกัน inverse relationship แต่ละ D จะได้ค่าที่ไม่ซ้ำกันแต่ละออปเจกของ C

| ฐานข้อมูลเชิงสัมพันธ์                              | ฐานข้อมูลเชิงวัตถุ                                     |
|----------------------------------------------------|--------------------------------------------------------|
| 1. ใช้ SQL-92                                      | 1. ภาษาทางด้านออปเจกจะช่วยลดขนาดของภาษาทางด้านดาต้าเบส |
| 2. เก็บข้อมูลในรูปของตาราง                         | 2. เก็บข้อมูลในรูปของอินสแตรนดาต้าไทป์                 |
| 3. ไม่รองรับ Complex Attribute                     | 3. รองรับ Complex Attribute                            |
| 4. ไม่สามารถสร้างโครงสร้างข้อมูลแบบใหม่ได้         | 4. สามารถสร้างโครงสร้างข้อมูลใหม่ได้                   |
| 5. เข้าถึงข้อมูลได้ง่าย และรวดเร็ว                 | 5. การเข้าถึงข้อมูลจะต้องผ่านเมธอดของคลาสเท่านั้น      |
| 6. ไม่รองรับความสามารถของออปเจก                    | 6. รองรับความสามารถของออปเจก                           |
| 7. เหมาะกับฐานข้อมูลประเภทโอเพอร์เรชั่นนอลดาต้าเบส | 7. เหมาะกับฐานข้อมูลขนาดกลาง                           |

ตารางที่ 5.2 ตารางเปรียบเทียบคุณสมบัติของฐานข้อมูลเชิงสัมพันธ์ กับฐานข้อมูลเชิงวัตถุ

### 5.3. โครงสร้างของภาษากำหนดรูปแบบเชิงวัตถุ

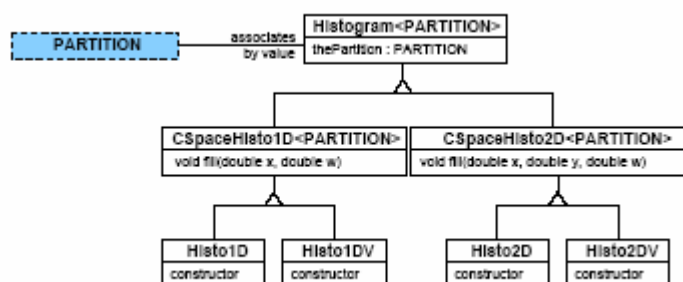
ระบบฐานข้อมูลเชิงวัตถุได้มีการพัฒนาอย่างต่อเนื่อง ซึ่งระบบฐานข้อมูลเชิงวัตถุนี้มีคุณสมบัติต่างๆ ที่สามารถรองรับข้อมูลที่ซับซ้อนได้ นอกจากนี้ยังมีองค์ประกอบต่างๆ ที่สามารถในงานขนาดใหญ่ได้ แต่เหตุผลประการหนึ่งที่ทำให้ระบบฐานข้อมูลเชิงวัตถุไม่ได้รับความนิยมใช้กันอย่างแพร่หลายก็คือ เพราะยังไม่มีมาตรฐานรองรับระบบฐานข้อมูลเชิงวัตถุนั่นเอง จึงได้มีความพยายามที่จะสร้างมาตรฐานให้กับฐานข้อมูลเชิงวัตถุนี้ โดยมีหน่วยงาน The Object Data Management Group (ODMG) มีหน้าที่ในการสร้างมาตรฐานให้กับระบบฐานข้อมูลเชิงวัตถุ นั่นคือภาษา ODL(Object Definition Language)

#### 5.3.1. ภาษากำหนดรูปแบบเชิงวัตถุ (Object Definition Language : ODL)

ในมาตรฐาน ODMG นี้ได้กำหนดรูปแบบภาษา ODL ไว้เพื่อเป็นภาษาในการสร้างออปเจกเพื่อนำไปใช้งานในระบบฐานข้อมูลอีกทีหนึ่ง มีรายละเอียดดังนี้

การสร้างคลาสใน ODQL ใน ODL คลาสจะถูกจัดให้อยู่ใน class family (class family คือ logical organization ของคลาส) อย่างเช่น คลาสที่เกี่ยวกับ user และ group อาจจัดอยู่ใน class family เดียวกัน class

family สามารถอ้างอิงขึ้นอปเจกใน class family อื่นได้ คลาสสามารถสืบทอดกันได้จากคลาภายใน class family เดียวกัน แต่สามารถมีแอททริบิวต์(property) ที่อ้างอิงอปเจกใน class family อื่น ดังรูปที่ 5-3



รูปที่ 5.3 ตัวอย่าง class family

เมื่อเรากำหนดคลาส เราจำเป็นต้องกำหนด Class family ด้วยคลาสหนึ่งๆ จะอยู่ใน class family ได้เพียงคลาสเดียวเท่านั้น ชื่อของคลาสจะต้อง unique ภายใน class family หนึ่งๆ แต่เราสามารถมีคลาสชื่อเดียวกันในคนละ class family ได้

ทุกๆ Persistent object ใน ODL จะต้องเป็นอินสแตรอนของคลาส และในการกำหนดคลาสนั้นจำเป็นต้องมี

- ชื่อคลาส
- Superclass : โดยจะมีหรือไม่ก็ได้ ทุกๆ user-defined class จะเป็น subclass ของ system class composite โดยอัตโนมัติ Superclass ที่กำหนดใน class-definition จะเป็น immediate superclass ของคลาสที่กำหนด
- Characteristic(method และ property) ของคลาส มีด้วยกัน 3 รูปแบบ
  1. Instance characteristic : ได้แก่ instance-level property และ method
  2. Class characteristic : ได้แก่ class-level property และ method โดย class-level property จะถูกใช้งานร่วมกัน โดยทุกๆ อินสแตรอนของคลาสนั้น เช่น คลาส User มีแอททริบิวต์ average\_salary ซึ่งเก็บเงินเดือนเฉลี่ยของพนักงานทุกคนในคลาสนั้น
  3. Method on collection : ODQL สนับสนุนเมธอดที่จะทำงานกับ collection ของ ออปเจก หรือของคลาส อย่างเช่น เมธอดที่ทำหน้าที่นับจำนวนของอินสแตรอนภายใน collection หรือเมธอดที่ให้อินสแตรอนกลับมา ตัวแรกและตัวสุดท้ายใน list และเมธอดที่ทำงานกับ collection of class นั้น คลาสใน collection ควรจะเป็นคลาสนั้น หรือเป็น Subclass ของคลาสนั้น

เราอาจกำหนดค่าเริ่มต้นให้กับ Class-level หรือ instance-level property ได้ และสำหรับ instance-level property นั้นเราสามารถกำหนด constraint อื่นๆ เช่น กำหนดให้แอททริบิวต์นั้นต้องมีค่า (not null) เป็นต้น การกำหนดเมธอดนั้น เราสามารถกำหนดการให้ค่าชนิดของข้อมูลกลับมาด้วย (return type) กำหนดชื่อของเมธอดและพารามิเตอร์ที่ต้องการ

## ตัวอย่างของ ODL ในการสร้างคลาส

## Interface Person

```

{
    /* instance properties */
    Attribute String Name;
    Attribute enum Gender{F,M};
    Attribute Date date_birth;
    /* instance operations */
    Integer Age();
}

```

## Interface Employee::Person

```

{
    /* instance properties */
    Attribute Unsigned Integer salary;
    Relationship Set<Skill> has_skills
        Inverse Skill::belongs_to;
    Relationship Department works_for
        Inverse Department::Employees;
    Relationship Department manages
        Inverse Department::Manager;
    /* instance operations */
    Unsigned Integer AvgSalary();
}

```

## Interface Skill

```

{
    /* instance properties */
    Attribute String name;
    Relationship Set<Employee> belongs_to
        Inverse Employee::has_skills;
}

```

## Interface Department

```

{
    /* instance properties */
    Attribute String name;
}

```



```

Attribute Integer budget;
Relationship Set<Employee> employees
    Inverse Employee::works_for;
Relationship Employee manager
    Inverse Employee::managers;
/* instance operations */
Unsigned Integer NumOfEmployee
}

```

### 1. การประกาศคลาส (Interface Declarations)

โดยคำว่า “Interface” ใน ODL ใช้ในการประกาศคลาส แล้วตามด้วยชื่อของคลาสที่เราต้องการประกาศ โดยในคลาสจะประกอบไปด้วยแอททริบิวต์, รีเลชันชิพ และเมธอด เราสามารถใช้งานแอททริบิวต์ที่เป็น atomic types หรือกำหนดแอททริบิวต์ให้เป็นชนิด structures, collections หรือ collection of structures โดยเราจะใช้คำว่า “Struct” ในการประกาศ ดังในตัวอย่างต่อไปนี้

```

Interface Star
{
    Attribute String name;
    Attribute Struct Addr
        { String street, String city } address;
}

```

### 2. การประกาศแอททริบิวต์ใน ODL (Attributes in ODL)

เป็นการประกาศชนิดของ Properties โดยใช้คำว่า “Attribute” ในการประกาศ เป็นการอธิบายคุณลักษณะของออบเจกต์ตัวนั้นๆ ตัวอย่างเช่น คลาส Person อาจจะประกอบด้วยแอททริบิวต์ name, birthday, address เป็นต้น

### 3. การประกาศความสัมพันธ์ใน ODL (Relationships in ODL)

ส่วนการประกาศความสัมพันธ์ระหว่างคลาสใน ODL เราจะใช้คำว่า “Relationship” ในการประกาศ และในทางกลับกัน เราก็กำหนด inverse relationship เพื่อเป็นแสดงความสัมพันธ์ย้อนกลับระหว่างความสัมพันธ์ของคลาส ซึ่งเราใช้คำว่า “Inverse” ในการอ้างถึงความสัมพันธ์ย้อนกลับนั้น โดยตามด้วยชื่อความสัมพันธ์ของอีกคลาสหนึ่งที่เราต้องการสร้าง inverse relationship เราสามารถแสดงตัวอย่างความสัมพันธ์ระหว่างคลาสได้ดังต่อไปนี้

```

Interface Movie
{

```

```

Attribute String title;
Attribute Integer year;
Attribute Integer length;
Attribute enum Film { color, blackAndWhite } filmType;
Relationship Set<Star> stars
    Inverse Star::starredIn;
Relationship Studio ownedBy
    Inverse Studio::owns;
}

Interface Star
{
    Attribute String name;
    Attribute Struct Addr
        { String street, String city } address;
    Relationship Set<Movie> starredIn
        Inverse Movie::stars;
}

Interface Studio
{
    Attribute String name;
    Attribute String address;
    Relationship Set<Movie> owns
        Inverse Movie::ownedBy;
}

```

#### รูปที่ 5.4 แสดงตัวอย่างรูปแบบของความสัมพันธ์ต่างๆ

#### 4. รูปแบบของความสัมพันธ์

ความสัมพันธ์ระหว่างคลาสมีด้วยกัน 3 รูปแบบ

1. **Many-many relationship** : เป็นความสัมพันธ์จากคลาส C ไปคลาส D โดยความสัมพันธ์เป็นเซตของ D ที่มีความเกี่ยวข้องกับแต่ละออบเจกต์ของ C ในทางกลับกัน inverse relationship ก็เป็นเซตของ C ที่มีความเกี่ยวข้องกับแต่ละออบเจกต์ของ D จากรูปที่ 5-4 stars เป็นความสัมพันธ์แบบ many-many จากคลาส Movie ไปคลาส Star และ starredIn เป็นความสัมพันธ์แบบ many-many จากคลาส Star ไปคลาส Movie โดยทั้งสองความสัมพันธ์ถูกกำหนดเป็นแบบ set

2. **Many-one relationship** : เป็นความสัมพันธ์จากคลาส C ไปคลาส D โดยความสัมพันธ์สำหรับแต่ละ C จะได้ค่าที่ไม่ซ้ำกันของ D ในทางกลับกัน inverse relationship จะมีเซตของ C ที่มีความเกี่ยวข้องกับแต่ละออบเจกต์ของ D จากรูปที่ 5-4 ownedBy เป็นความสัมพันธ์แบบ many-one จากคลาส Movie ไปคลาส Studio เราต้องกำหนด inverse relationship เป็นความสัมพันธ์แบบ one-many จากคลาส D ไปคลาส C โดยในที่นี้ owns เป็นความสัมพันธ์แบบ one-many จากคลาส Studio ไปคลาส Movie

3. **One-one relationship** : เป็นความสัมพันธ์จากคลาส C ไปคลาส D โดยความสัมพันธ์สำหรับแต่ละ C จะได้ค่าที่ไม่ซ้ำกันของ D ในทางกลับกัน inverse relationship แต่ละ D จะได้ค่าที่ไม่ซ้ำกันแต่ละออบเจกต์ของ C

## 5. ชนิดของตัวแปร (Types in ODL) แบ่งได้ 2 แบบ

1. Atomic types : ได้แก่ integer, float, character, character string, boolean และ enumerations สำหรับตัวแปรชนิด enumeration นั้น เป็นการกำหนดรูปแบบของค่าหรือข้อความให้มีค่าเท่ากับจำนวนในแบบตัวเลข

2. Interface types : เป็นตัวแปรที่อ้างอิงถึงออบเจกต์ของคลาส ตัวอย่างเช่น ในคลาสหนึ่งมีส่วนประกอบของอีกคลาสหนึ่งอยู่ด้วย

### 5.3.2.ภาษากำหนดรูปแบบ Cache' (Cache' Definition Language: CDL)

CDL เป็นรูปแบบของไฟล์ที่ใช้ในการสร้างคลาสของระบบฐานข้อมูล Cache' ซึ่งเป็นฐานข้อมูลแบบเชิงวัตถุประเภทหนึ่ง ซึ่งเราสามารถใช้ text edit ในการสร้างไฟล์นี้ ในไฟล์ CDL เราสามารถกำหนดข้อความเพื่อใช้ในการอธิบาย (comment) ได้ 2 แบบดังนี้

1. // หมายถึง เราไม่ต้องการให้โปรแกรมพิจารณาคำสั่งด้านหลังของเครื่องหมาย //
  2. /\* \*/ ใช้สำหรับเมื่อเราไม่ต้องการให้โปรแกรมพิจารณาคำสั่งระหว่างเครื่องหมาย /\* กับ \*/
- เมื่อเราเขียนคำสั่งตรงตามรูปแบบของไฟล์ CDL เราจะต้องมีสัญลักษณ์ที่ใช้ในการจบคำสั่งด้วยสัญลักษณ์; (semicolon) ส่วนรูปแบบของคำสั่งประกอบด้วยการประกาศที่ใช้งานหลักๆ ดังนี้

#### 1. การประกาศคลาส (Definition Classes)

มีรูปแบบการประกาศคลาส ดังนี้

```
Class {package name.}class name { keywordspecification;... }
```

โดย Class name คือชื่อของคลาสที่เราต้องการประกาศ

Package name คือชื่อของที่เราประกาศใช้งานให้กับคลาส โดยถ้าชื่อ package นั้นเราไม่ได้ใส่ค่าเข้าไป ตัวแอปพลิเคชันจะกำหนดให้ค่าปกติ (default) มีค่าเท่ากับ User.

keywordspecification คือรูปแบบการกำหนดค่าต่างๆ ให้กับคลาสของเรา โดยมีรูปแบบต่างๆ ที่ใช้ในการอธิบายคลาสที่เราสร้าง

#### 2. การประกาศตัวแปร(Definition of Properties)

เป็นส่วนที่เราใช้ในการประกาศแอททริบิวต์ โดยมีรูปแบบการประกาศแอททริบิวต์ ดังนี้

Attribute ชื่อแอททริบิวต์ { รายละเอียดของแอททริบิวต์ }

โดยในส่วนของรายละเอียดของแอททริบิวต์ เราสามารถกำหนดรูปแบบต่างๆ ของแอททริบิวต์ได้ ดังต่อไปนี้

- 1) Calculated : มีความหมายว่า เราต้องคำนวณค่าให้กับแอททริบิวต์นี้ เนื่องจากแอททริบิวต์ไม่ได้เก็บข้อมูลไว้ในหน่วยความจำโดยตรง
- 2) Initial = expression : เป็นการกำหนดค่าเริ่มต้นให้กับตัวแอททริบิวต์
- 3) Private : ใช้กำหนดสิทธิในการเข้าถึงค่าแอททริบิวต์
- 4) Required : เป็นการบอกให้ทราบว่าแอททริบิวต์นี้จะต้องมีค่า(not null)
- 5) Type = datatype : ใช้สำหรับอ้างอิงชนิดของข้อมูลให้กับแอททริบิวต์

### 3. การประกาศเมธอด(Definition of Methods)

เป็นส่วนที่ใช้ตอบสนองต่อข่าวสารที่เรียกใช้งานออบเจกต์ของคลาส โดยมีรูปแบบการประกาศเมธอด ดังนี้

Method ชื่อเมธอด {รายละเอียดของเมธอด}

โดยในส่วนของรายละเอียดของเมธอด เราสามารถกำหนดรูปแบบต่างๆ ของเมธอดได้ดังต่อไปนี้

- 1) Call = routinename : ใช้ในการเรียกใช้เมธอดให้ทำงาน
- 2) Final : ใช้ประกาศเพื่อบอกว่าเมธอดนี้ไม่สามารถเปลี่ยนแปลงส่วนของการจัดการได้อีกแล้ว (Overriding)
- 3) Expression = expression : ใช้เรียกฟังก์ชันของ Cache' ObjectScript
- 4) Private : ใช้กำหนดสิทธิในการเรียกใช้งานเมธอด
- 5) Returntype = datatype : ใช้กำหนดค่าชนิดของข้อมูลที่ต้องการส่งค่าออกมาจากเมธอด

ตัวอย่างรูปแบบคลาสของไฟล์ CDL

Class User.Person

```
{
    description = "General person class";
    super = %Library.Persistent;
    persistent;
    /* Definition of the properties */
    attribute Name { type = %Library.String; required; }
    attribute Firstname { type = %Library.String; }
    attribute DOB { type = %Library.Date; }
    attribute Age { type = %Library.Integer; calculated;
```

```
sqlcompute =
{
    :Set {AgeInYears}=$Horolog-{DOB}\365.2425;
}
sqlcompute; sqlfieldname = AgeInYear; }

/* Definition of the methods */
method AgeGet()
{
    returntype = %Library.Integer;
    expression = $Horolog-..DOB\365.2425;
}
}
```